
UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

Facultad de Ingeniería

Ingeniería en Computación

Documento de Entrega

Aplicaciones Web Escalables (224501)

Reporte de Implementación

KitchenFlow: Control de Producción y Costeo para Cafeterías

Alumno: Miguel Alejandro Gutiérrez Silva

Profesor: Ing. Francisco Javier Gómez Vázquez

Fecha: Semestre 2025-2026/II

1. Propósito del documento

Este documento resume el estado de implementación real de **KitchenFlow** con base en la propuesta obligatoria contenida en `report/propuesta/reporte_template.tex`. El objetivo no es señalar cambios normales de diseño o de stack como defectos, sino distinguir con claridad entre:

- funcionalidades que sí quedaron implementadas;
- funcionalidades que cambiaron de forma, pero conservan el objetivo de negocio;
- elementos que se ajustaron deliberadamente por razones prácticas de alcance o arquitectura.

2. Resumen ejecutivo

KitchenFlow evolucionó desde una propuesta visual a una aplicación funcional que ya cubre el flujo operativo principal de una cafetería:

abastecimiento → recetas → producción → ventas → dashboard

Además, el sistema ya incluye autenticación basada en JWT, control de acceso por roles y una pantalla administrativa para gestión de usuarios. A nivel general, el proyecto **sí implementa el núcleo funcional planteado en la propuesta**. Las diferencias principales respecto al documento original se concentran en decisiones de arquitectura y en ajustes de experiencia de usuario, no en la ausencia del flujo central de negocio.

3. Matriz de cobertura funcional

Elemento de la propuesta	Cobertura actual	Estado	Observación
Autenticación JWT	Login, sesión actual, restauración de sesión y protección de API/webapp	Implementado	Se implementó autenticación real basada en token y validación de sesión.
RBAC (Administrador, Cocina, Piso)	Roles predefinidos, guards frontend y middleware backend	Implementado	El acceso está restringido por rol tanto en rutas Angular como en endpoints Express.

Elemento de la propuesta	Cobertura actual	Estado	Observación
Gestión de usuarios por administrador	Alta, edición, activación/desactivación y restablecimiento de contraseña	Implementado	Se agregó un módulo administrativo para completar este punto de la propuesta.
Recetario y costeo	CRUD de recetas, costo total, margen y estado de rentabilidad	Implementado	El módulo ya funciona con datos reales y persiste costos calculados.
Costo Promedio Ponderado (WAC)	Registro de compras y recálculo inmediato de costo promedio por insumo	Implementado	El flujo de abastecimiento actualiza stock y costo promedio ponderado.
Producción con explosión de materiales	Órdenes con estados, reserva de insumos, cierre con consumo real y merma por lote	Implementado	La producción ya no es mockup; es una operación con control de inventario y resultados reales.
Control de mermas	Merma integrada en el cierre de producción mediante consumo real vs. planeado y resumen de desperdicio	Implementado con ajustes	No existe como módulo independiente, pero la información requerida sí se registra y se usa en analítica.
Despacho / ventas en piso	Registro real de ventas, decremento de stock terminado y ticket persistido	Implementado	El personal de piso puede operar sobre producto terminado disponible.
Dashboard de rentabilidad	Indicadores reales de ventas, producción, compras, alertas y productos críticos	Implementado	El dashboard consume datos reales del backend, no datos mockeados.

Elemento de la propuesta	Cobertura actual	Estado	Observación
Mockups principales	Dashboard, recetas, producción, ventas y abastecimiento reimplementados como pantallas funcionales	Implementado	Las pantallas propuestas ya están representadas como módulos operativos.
Conversión de unidades	Catálogo uniforme de unidades, sin motor genérico de conversión entre unidades heterogéneas	Implementado con ajustes	Se normalizó la captura de unidades para evitar complejidad adicional innecesaria en el alcance actual.
Auditoría operativa de producción	Historial por orden con estados, tiempos, cantidades reales y costos de merma	Implementado	El flujo de producción conserva trazabilidad suficiente para revisión posterior.

4. Cambios entre propuesta e implementación

La tabla siguiente resume **qué cambió, qué se implementó realmente y por qué ese cambio fue razonable dentro del desarrollo del proyecto.**

Tema	Propuesta original	Implementación final	Motivo del ajuste
Stack backend	Hono + ZenStack/Prisma + PostgreSQL 16	Express + Mongoose + MongoDB	Se decidió consolidar la implementación sobre el stack realmente utilizado por el proyecto y por el entorno de despliegue. El cambio es de arquitectura, no de objetivo funcional.
Contratos compartidos	RPC/DTOs derivados de Zod	OpenAPI + cliente tipado generado para Angular	Se mantuvo la idea de contrato tipado compartido, pero usando OpenAPI como mecanismo más compatible con el stack real.

Tema	Propuesta original	Implementación final	Motivo del ajuste
Abastecimiento	Wizard de 4 pasos	Tabla operativa de insumos + modales de compra e historial	Se privilegió una interacción más útil para uso real diario, manteniendo el propósito del módulo.
Recetas	Editor de receta tipo maestro-detalle	Editor modal con costo, margen y persistencia real	Se conservó el objetivo del mockup, pero con una UI ajustada al patrón final del sistema.
Producción	Registro atómico simple de lote	Órdenes con estados, apartados, cierre real y merma asociada	El modelo evolucionó para reflejar mejor la operación real de cocina y el control de recursos.
Mermas	Módulo explícito en propuesta	Información absorbida en cierre de producción y dashboard	Para el alcance final, la merma quedó integrada al flujo donde realmente se origina la desviación.
Ventas	Mockup tipo terminal táctil	Tabla de productos vendibles + modal de ticket y ventas recientes	Se simplificó la interacción sin perder el despacho inmediato de producto terminado.
Dashboard	KPIs y comparativas mensuales	KPIs reales, tendencias, alertas, productos críticos y actividad reciente	Se priorizó mostrar información accionable basada en datos ya disponibles del sistema.
Usuarios	Gestión administrativa implícita en el rol ADMIN	Pantalla específica de gestión de usuarios	Se agregó para cerrar una ausencia funcional importante respecto a la propuesta.
Conversión de unidades	Conversión explícita entre unidad de compra y unidad de consumo	Catálogo uniforme de unidades capturadas de forma operativa	Se acotó el problema usando normalización desde la captura, suficiente para el alcance académico implementado.

5. Arquitectura implementada

La arquitectura final de KitchenFlow se consolidó como una solución desacoplada de dos capas principales:

- **Frontend Angular 21:** interfaz de usuario con rutas lazy, guards por rol, cliente tipado generado desde OpenAPI y componentes standalone bajo un sistema visual local inspirado en ZardUI.
- **Backend Express + MongoDB:** API REST con autenticación JWT, middleware de autorización por rol, lógica de negocio para inventario, recetas, producción, ventas, usuarios y analítica.

Aunque la propuesta original planteaba Hono, ZenStack, Prisma y PostgreSQL, la implementación real se estabilizó sobre **Express + Mongoose + MongoDB**. Este cambio permitió trabajar sobre el stack que efectivamente estaba en uso dentro del repositorio, reduciendo fricción de integración y conservando el objetivo principal de la propuesta: un sistema funcional, tipado y desplegable.

En términos de intercambio de datos, la integridad entre frontend y backend se resolvió mediante:

- **OpenAPI** como contrato formal de la API;
- **generación de tipos** para Angular a partir del esquema OpenAPI;
- **validación operativa por capas** a través de controladores, modelos y middleware del backend.

Esta arquitectura implementada mantiene el espíritu de la propuesta original al conservar separación de responsabilidades, tipado entre capas y control explícito del dominio de negocio.

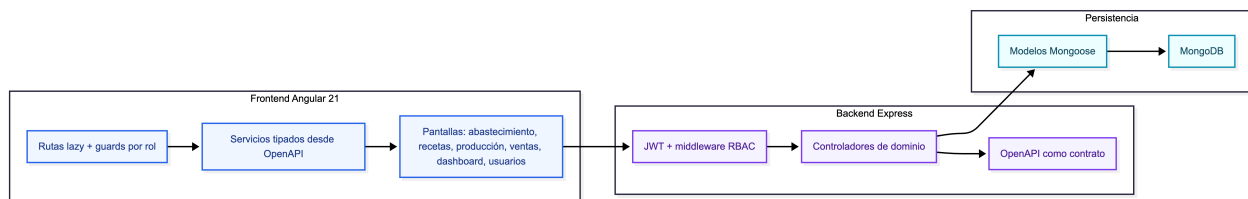


Figura 1: Arquitectura implementada de KitchenFlow.

6. Dockerización

El proyecto fue preparado para ejecutarse de forma consistente mediante contenedores Docker coordinados por `docker-compose.yml`. La composición actual contempla tres servicios:

- **frontend:** aplicación Angular servida en desarrollo con hot reload;
- **backend:** API Express con reinicio automático durante desarrollo;
- **mongo:** instancia de MongoDB para persistencia del sistema.

Esta decisión aporta varias ventajas prácticas:

- unifica el entorno de desarrollo entre máquinas;
- reduce diferencias entre entorno local y entorno desplegado;
- simplifica el arranque del proyecto completo;

- facilita la futura reproducción o evaluación del sistema.

Además, la aplicación fue configurada para que el frontend consuma la API bajo la misma solución desplegada, conservando una estructura clara de puertos y servicios. En términos académicos y operativos, la dockerización permitió demostrar no sólo la lógica del sistema, sino también su **capacidad real de puesta en marcha**.

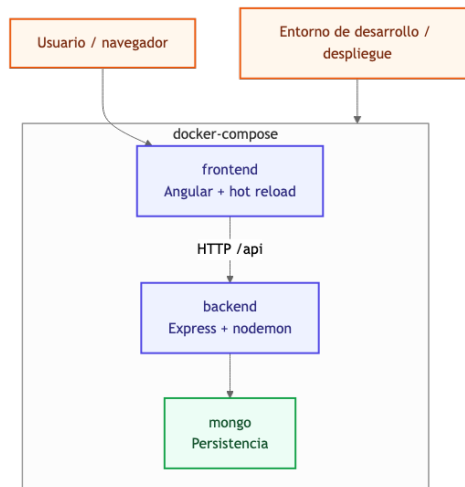


Figura 2: Esquema de dockerización usado en el proyecto.

7. CI/CD y despliegue

El proyecto también incorpora una estrategia de integración y despliegue continuo alineada con la implementación final. Se configuró un flujo de despliegue utilizando **GitHub Actions** con un **runner self-hosted** que se ejecuta en un **VPS propio**.

Esto implica que:

- el repositorio puede disparar procesos automáticos de despliegue;
- el runner no depende de infraestructura efímera externa;
- la construcción y levantamiento del sistema ocurren directamente sobre el servidor de destino.

En términos prácticos, el pipeline permite:

- obtener el código más reciente del repositorio;
- reconstruir los servicios necesarios;
- levantar el sistema mediante Docker Compose;
- verificar la disponibilidad básica del frontend y del backend.

El hecho de contar con un **VPS propio que también hospeda el runner** fortalece el carácter de entrega del proyecto, ya que no se trata únicamente de una implementación local o académica, sino de una solución que fue preparada para correr en infraestructura controlada por el autor.

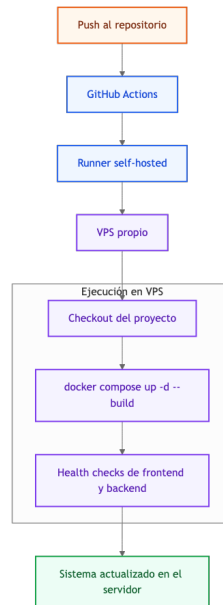


Figura 3: Flujo de CI/CD con runner self-hosted ejecutándose en VPS propio.

8. Capturas del sistema

Como evidencia complementaria de la implementación, se incluyen a continuación capturas reales de la interfaz operativa de KitchenFlow. Las imágenes fueron generadas directamente sobre la aplicación en ejecución y muestran el estado actual de los módulos principales.

9. Módulos implementados

9.1) Autenticación y autorización

El sistema implementa autenticación con JWT, recuperación de sesión y protección de acceso por rol. Existen tres perfiles operativos:

- **Administrador:** acceso a dashboard, usuarios, abastecimiento, recetas, producción y ventas.
- **Cocina:** acceso a recetas y producción.
- **Piso:** acceso a ventas.

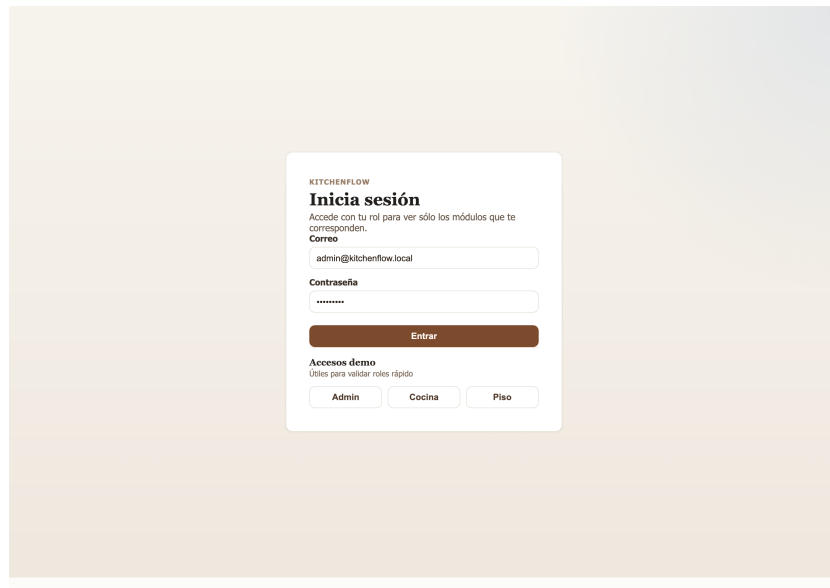


Figura 4: Pantalla de inicio de sesión de KitchenFlow. Se muestran accesos diferenciados por rol para administrador, cocina y piso de ventas.

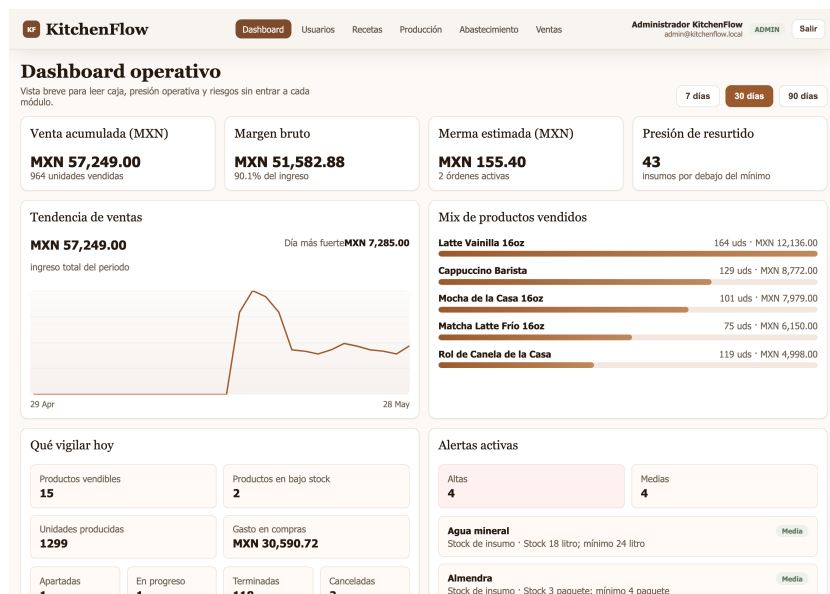


Figura 5: Dashboard operativo con indicadores reales de ventas, margen, merma, resurtido, tendencias y alertas de operación.

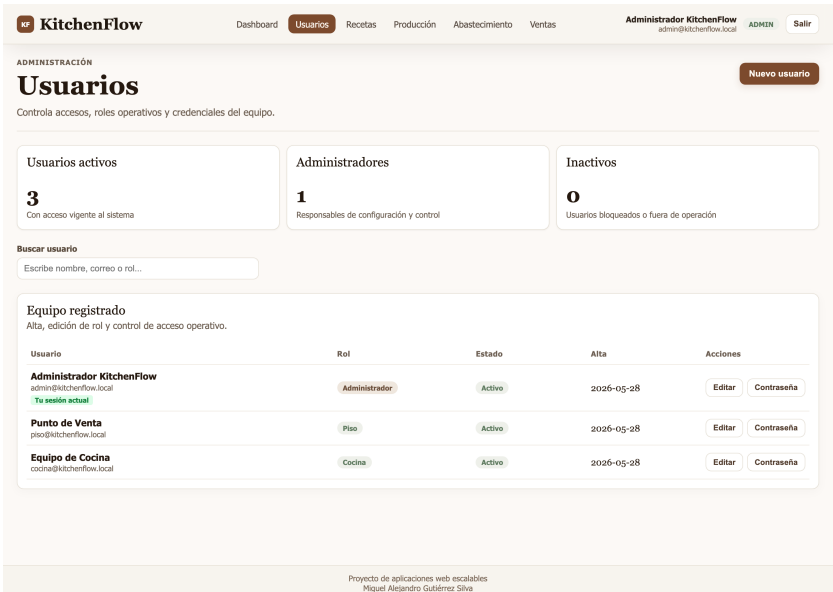


Figura 6: Módulo de gestión de usuarios. El administrador puede consultar cuentas activas, altas recientes, roles y acciones de mantenimiento.

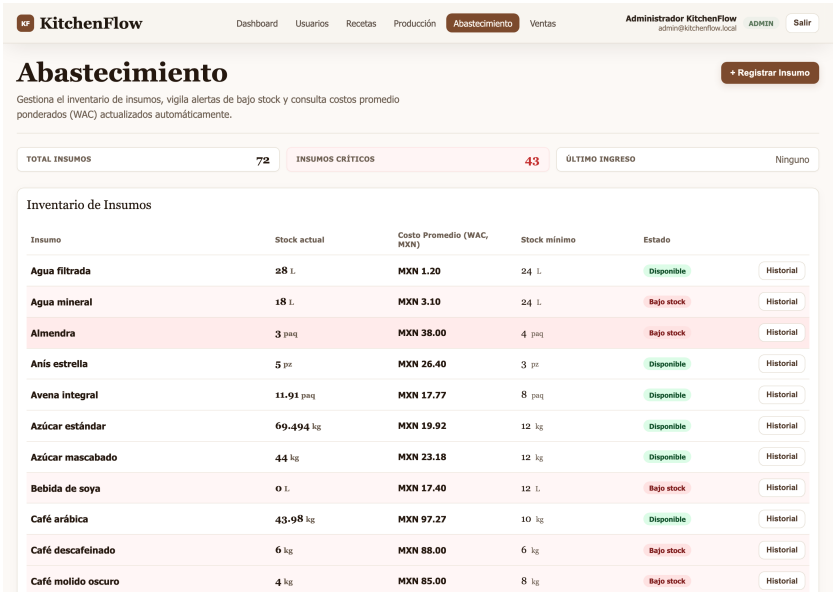


Figura 7: Módulo de abastecimiento con inventario de insumos, costo promedio ponderado, stock mínimo y control de historial de compras.

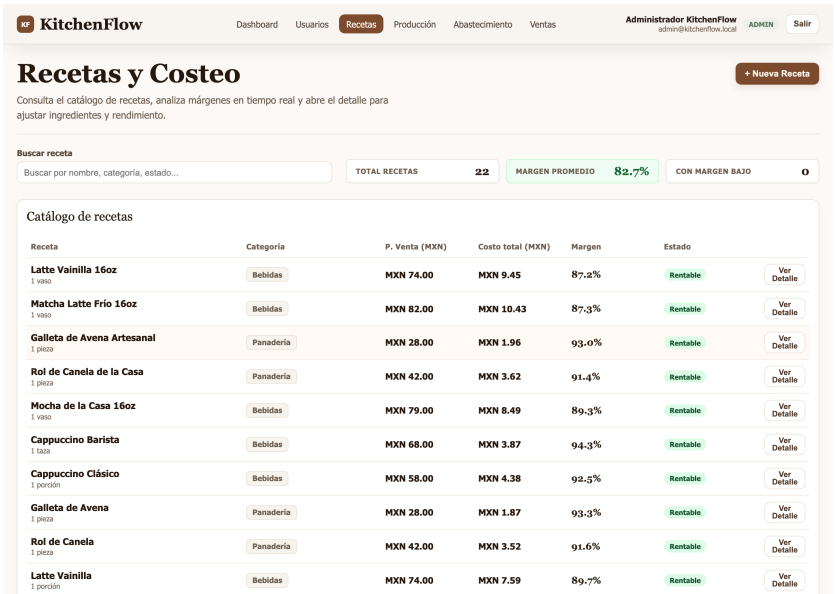


Figura 8: Pantalla de recetas y costeo. Se muestran catálogo, costos unitarios, precio de venta, margen y estado de rentabilidad por producto.

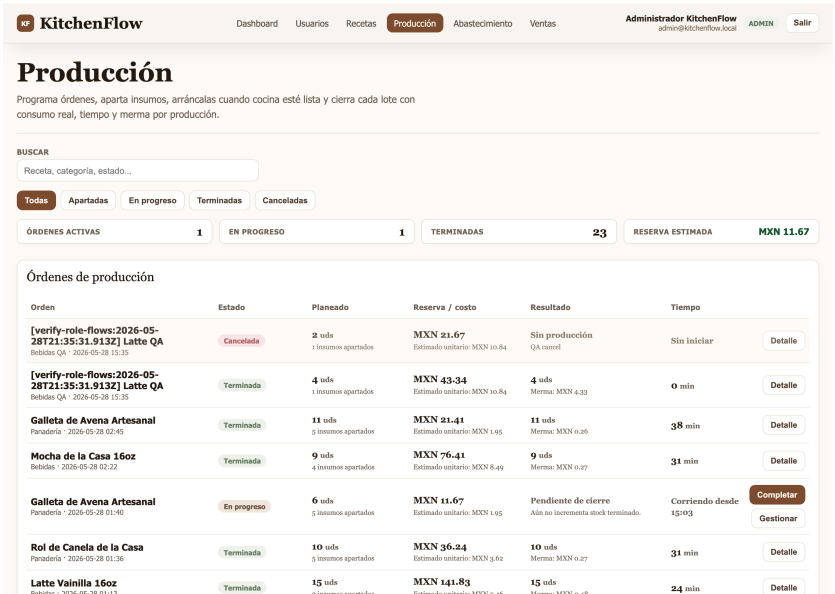


Figura 9: Módulo de producción con órdenes por estado, costos planeados, resultados reales y seguimiento operativo del lote.

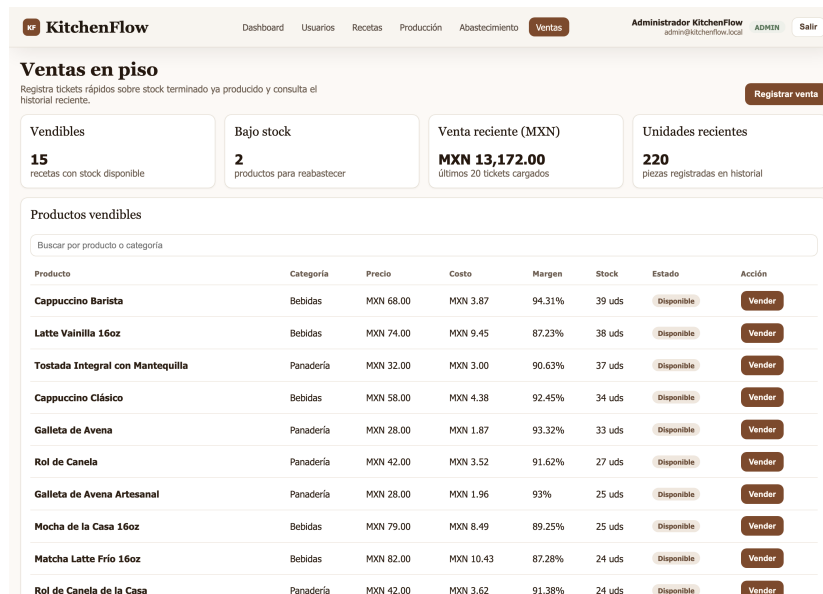


Figura 10: Pantalla de ventas en piso con productos vendibles, stock disponible, ticket rápido e historial reciente de ventas.

9.2) Abastecimiento

El módulo de abastecimiento permite registrar insumos, consultar su stock mínimo, revisar historial de compras y registrar recepciones con recálculo de costo promedio ponderado.

9.3) Recetas y costeo

Las recetas almacenan insumos, cantidades, costo total, precio de venta y margen. El sistema permite evaluar rentabilidad por receta y mantener el stock terminado asociado al producto.

9.4) Producción

La producción fue implementada como un flujo con estados operativos:

- PENDING
- IN PROGRESS
- COMPLETED
- CANCELLED

Durante este flujo se apartan insumos, se valida disponibilidad, se registra consumo real y se calcula el costo de merma por lote.

9.5) Ventas

El módulo de ventas registra tickets reales, descuenta stock terminado y conserva snapshots de precio, costo y margen por línea de venta.

9.6) Dashboard

El dashboard presenta indicadores reales de operación: ventas, margen bruto, compras, merma, productos más vendidos, alertas de inventario y actividad reciente de producción.

9.7) Gestión de usuarios

Se implementó una pantalla administrativa para alta, edición de rol, activación/desactivación y restablecimiento de contraseña. Con esto se cubre la responsabilidad operativa del rol administrador definida en la propuesta.

10. Elementos no críticos ajustados por alcance

No se identifican ausencias funcionales críticas de primer orden respecto al alcance aprobado del proyecto. Los ajustes pendientes o simplificados se concentran en decisiones de alcance controlado:

- la **conversión genérica de unidades** no fue desarrollada como motor separado, ya que el sistema trabaja con un catálogo normalizado de captura;
- el manejo de **mermas** no vive en una pantalla exclusiva, pero la información sí se registra y se usa en producción y dashboard;
- la arquitectura final no reproduce literalmente el stack declarado en la propuesta, pero sí resuelve los mismos objetivos funcionales.

11. Conclusión

La implementación final de **KitchenFlow** conserva el propósito central de la propuesta: controlar insumos, costear recetas, transformar materia prima en producto terminado, registrar ventas y exponer indicadores de rentabilidad para la toma de decisiones.

Las diferencias detectadas frente al documento original se explican principalmente por:

- decisiones de arquitectura más adecuadas al stack real del repositorio;
- mejoras de flujo de trabajo en la interfaz;
- integración de ciertas funciones en módulos más operativos.

En términos de entrega, el proyecto puede considerarse **funcionalmente implementado en su núcleo principal**. Las variaciones observadas son, en su mayoría, ajustes razonables de implementación y no omisiones graves del objetivo del sistema.